PRIYANSHU SHARMA (23/38007)

1. **Write a program to perform encryption and decryption using Caesar cipher (substitutional cipher).**

```python
def encrypt(text, shift):
    result = ""

    for char in text:
        if char.isupper():
            result += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
        elif char.islower():
            result += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
        else:
            result += char

    return result

def decrypt(text, shift):
    return encrypt(text, 26 - shift)

text = "Hello World"
shift = 3

print(f"Original text: {text}")

encrypted = encrypt(text, shift)
print(f"Encrypted text: {encrypted}")

decrypted = decrypt(encrypted, shift)
print(f"Decrypted text: {decrypted}")
```

OUTPUT:

```
Original text: Hello World
Encrypted text: Khoor Zruog
Decrypted text: Hello World
```

2. **Write a program to perform encryption and decryption using Rail Fence Cipher (transpositional cipher).**

```
def encrypt(text, rails):
    if rails == 1:
        return text

    fence = [''] * rails
    rail = 0
    direction = 1

    for char in text:
        fence[rail] += char
        rail += direction

        if rail == 0 or rail == rails - 1:
            direction = -direction

    return ''.join(fence)

def decrypt(text, rails):
    if rails == 1:
        return text

    fence = [0] * rails
    rail = 0
    direction = 1

    for i in range(len(text)):
        fence[rail] += 1
        rail += direction

        if rail == 0 or rail == rails - 1:
            direction = -direction

    rows = []
    index = 0
    for count in fence:
        rows.append(text[index:index + count])
        index += count

    result = ''
    rail = 0
    direction = 1
    pos = [0] * rails

    for i in range(len(text)):
        result += rows[rail][pos[rail]]
        pos[rail] += 1
        rail += direction

        if rail == 0 or rail == rails - 1:
            direction = -direction
```

```
    return result

text = "HELLO WORLD"
rails = 3

print(f"Original text: {text}")

encrypted = encrypt(text, rails)
print(f"Encrypted text: {encrypted}")

decrypted = decrypt(encrypted, rails)
print(f"Decrypted text: {decrypted}")
```

OUTPUT:

```
Original text: HELLO WORLD
Encrypted text: HOREL OLLWD
Decrypted text: HELLO WORLD
```

3. **Write a Python program that defines a function and takes a password string as input and returns its SHA-256 hashed representation as a hexadecimal string.**

```
import hashlib

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

password = input("Enter password: ")
hashed = hash_password(password)
print(f"SHA-256 Hash: {hashed}")
```

OUTPUT:

```
Enter password: priyanshusharmaisthebest1474
SHA-256 Hash: 4e54f302274e383cf4d2689a6371ff1160af13c5b7c97102ebdaa579ad86e9ff
```

4. **Write a Python program that reads a file containing a list of usernames and passwords, one pair per line (separated by a comma). It checks each password to see if it has been leaked in a data breach. You can use the "Have I Been Pwned" API (https://haveibeenpwned.com/API/v3) to check if a password has been leaked.**

```python
import hashlib
import requests

def check_password_leaked(password):
    sha1_hash = hashlib.sha1(password.encode()).hexdigest().upper()
    prefix = sha1_hash[:5]
    suffix = sha1_hash[5:]

    url = f"https://api.pwnedpasswords.com/range/{prefix}"
    response = requests.get(url)

    if response.status_code != 200:
        return None

    hashes = response.text.splitlines()
    for line in hashes:
        hash_suffix, count = line.split(':')
        if hash_suffix == suffix:
            return int(count)

    return 0

def check_file(filename):
    try:
        with open(filename, 'r') as f:
            for line in f:
                line = line.strip()
                if not line:
                    continue

                parts = line.split(',')
                if len(parts) != 2:
                    print(f"Invalid format: {line}")
                    continue

                username, password = parts[0].strip(), parts[1].strip()
                count = check_password_leaked(password)

                if count is None:
                    print(f"{username}: Error checking password")
                elif count > 0:
                    print(f"{username}: LEAKED ({count} times)")
                else:
                    print(f"{username}: Safe")

    except FileNotFoundError:
        print(f"File '{filename}' not found")
```

```
filename = input("Enter filename (e.g., passwords.txt): ")
check_file("passwords.txt")
```

OUTPUT:

```
Enter filename (e.g., passwords.txt): passwords.txt
john: LEAKED (2031380 times)
alice: LEAKED (21969901 times)
bob: LEAKED (1138064 times)
sarah: LEAKED (116434 times)
mike: LEAKED (180041686 times)
emma: LEAKED (5402944 times)
david: LEAKED (41213657 times)
lisa: LEAKED (669691 times)
```

5. **Write a Python program that generates a password using a random combination of words from a dictionary file.**

```python
import random

def generate_password(dict_file, num_words=4):
    try:
        with open(dict_file, 'r') as f:
            words = [line.strip() for line in f if line.strip()]

        if len(words) < num_words:
            print("Not enough words in dictionary")
            return None

        selected = random.sample(words, num_words)
        return '-'.join(selected)

    except FileNotFoundError:
        print(f"Dictionary file '{dict_file}' not found")
        return None

dict_file = input("Enter dictionary file (e.g., words.txt): ")
num_words = int(input("Enter number of words (default 4): ") or "4")

password = generate_password(dict_file, num_words)
if password:
    print(f"Generated password: {password}")
```

OUTPUT:

```
Enter dictionary file (e.g., words.txt): dictionary.txt
Enter number of words (default 4): 4
Generated password: silver-cascade-diamond-dragon
```

6. **Write a Python program that simulates a brute-force attack on a password by trying out all possible character combinations.**

```python
import itertools
import string
import time

def brute_force(target_password, charset, max_length):
    attempts = 0
    start_time = time.time()

    for length in range(1, max_length + 1):
        for attempt in itertools.product(charset, repeat=length):
            attempts += 1
            guess = ''.join(attempt)

            if attempts % 10000 == 0:
                print(f"Attempts: {attempts}, Current: {guess}")

            if guess == target_password:
                elapsed = time.time() - start_time
                print(f"\nPassword found: {guess}")
                print(f"Attempts: {attempts}")
                print(f"Time: {elapsed:.2f} seconds")
                return True

    print(f"\nPassword not found after {attempts} attempts")
    return False

target = input("Enter target password: ")
max_len = int(input("Enter max length to try: "))

print("\nCharset options:")
print("1. Digits only (0-9)")
print("2. Lowercase letters (a-z)")
print("3. Alphanumeric (a-z, 0-9)")
print("4. All printable characters")

choice = input("Select charset (1-4): ")

if choice == '1':
    charset = string.digits
elif choice == '2':
    charset = string.ascii_lowercase
elif choice == '3':
    charset = string.ascii_lowercase + string.digits
else:
    charset = string.ascii_letters + string.digits + string.punctuation

print(f"\nStarting brute force attack...")
print(f"Charset: {charset}")
print(f"Max length: {max_len}\n")
```

```
brute_force(target, charset, max_len)
```

OUTPUT 1:

```
Enter target password: 12345
Enter max length to try: 5

Charset options:
1. Digits only (0-9)
2. Lowercase letters (a-z)
3. Alphanumeric (a-z, 0-9)
4. All printable characters
Select charset (1-4): 1

Starting brute force attack...
Charset: 0123456789
Max length: 5

Attempts: 10000, Current: 8889
Attempts: 20000, Current: 08889

Password found: 12345
Attempts: 23456
Time: 4.72 milliseconds
```

OUTPUT 2:

```
Enter target password: cat
Enter max length to try: 3

Charset options:
1. Digits only (0-9)
2. Lowercase letters (a-z)
3. Alphanumeric (a-z, 0-9)
4. All printable characters
Select charset (1-4): 2

Starting brute force attack...
Charset: abcdefghijklmnopqrstuvwxyz
Max length: 3


Password found: cat
Attempts: 2074
Time: 0.00 milliseconds
```

OUTPUT 3:

```
Enter target password: ab12
Enter max length to try: 4

Charset options:
1. Digits only (0-9)
2. Lowercase letters (a-z)
3. Alphanumeric (a-z, 0-9)
4. All printable characters
Select charset (1-4): 3

Starting brute force attack...
Charset: abcdefghijklmnopqrstuvwxyz0123456789
Max length: 4

Attempts: 10000, Current: gy1
Attempts: 20000, Current: oot
Attempts: 30000, Current: wel
Attempts: 40000, Current: 34d
Attempts: 50000, Current: abt5

Password found: ab12
Attempts: 50285
Time: 12.00 milliseconds
```

OUTPUT 4:

```
Enter target password: a#
Enter max length to try: 2

Charset options:
1. Digits only (0-9)
2. Lowercase letters (a-z)
3. Alphanumeric (a-z, 0-9)
4. All printable characters
Select charset (1-4): 4

Starting brute force attack...
Charset: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
Max length: 2


Password found: a#
Attempts: 159
Time: 0.00 milliseconds
```

7. Demonstrate the usage/sending of a digitally signed document.

```
# ============================================================================
# PROGRAM 7: Digital Signature - Demonstrate Sending/Receiving Signed Document
# ============================================================================

from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.backends import default_backend
import base64

class DigitalSignature:
    def __init__(self):
        # Generate RSA key pair
        self.private_key = rsa.generate_private_key(
            public_exponent=65537,
            key_size=2048,
            backend=default_backend()
        )
        self.public_key = self.private_key.public_key()

    def sign_document(self, document):
        """Sign a document using private key"""
        signature = self.private_key.sign(
            document.encode(),
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )
        return base64.b64encode(signature).decode()

    def verify_signature(self, document, signature):
        """Verify document signature using public key"""
        try:
            self.public_key.verify(
                base64.b64decode(signature),
                document.encode(),
                padding.PSS(
                    mgf=padding.MGF1(hashes.SHA256()),
                    salt_length=padding.PSS.MAX_LENGTH
                ),
                hashes.SHA256()
            )
            return True
        except Exception:
            return False

    def export_public_key(self):
        """Export public key for sharing"""
        pem = self.public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
```

```
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        )
        return pem.decode()

# Demonstration
print("=" * 70)
print("DIGITAL SIGNATURE DEMONSTRATION")
print("=" * 70)

# Sender creates digital signature system
sender = DigitalSignature()
print("\n[SENDER] Generated RSA key pair")

# Document to be signed
document = input("\nEnter document content to sign: ")

# Sign the document
signature = sender.sign_document(document)
print(f"\n[SENDER] Document signed successfully!")
print(f"Signature: {signature[:50]}...")

# Export public key (to be shared with receiver)
public_key_pem = sender.export_public_key()
print(f"\n[SENDER] Public key exported (to be shared with receiver)")

# Simulate sending document and signature
print("\n" + "=" * 70)
print("SENDING: Document + Signature to Receiver")
print("=" * 70)

# Receiver verifies the signature
print("\n[RECEIVER] Received document and signature")
is_valid = sender.verify_signature(document, signature)

if is_valid:
    print("✓ Signature VERIFIED - Document is authentic and unmodified")
    print("✓ Document integrity confirmed")
    print("✓ Sender identity authenticated")
else:
    print("✗ Signature INVALID - Document may be tampered or from untrusted source")

# Test with tampered document
print("\n" + "=" * 70)
print("TESTING: Tampered Document Detection")
print("=" * 70)
tampered_doc = document + " [TAMPERED]"
print(f"\n[ATTACKER] Modified document: {tampered_doc}")

is_valid_tampered = sender.verify_signature(tampered_doc, signature)
if is_valid_tampered:
    print("✓ Signature verified (This should NOT happen)")
else:
    print("✗ Signature INVALID - Tampering detected!")
```

OUTPUT:

```
======================================================================
DIGITAL SIGNATURE DEMONSTRATION
======================================================================

[SENDER] Generated RSA key pair

Enter document content to sign: This is a confidential contract agreement between Party A and Party B wort
h $50,000

[SENDER] Document signed successfully!
Signature: HesH8JG24l6EF8Eg4SW2qgmRL9ovFnHYQp2tXkRidMm4b60Slz...

[SENDER] Public key exported (to be shared with receiver)

======================================================================
SENDING: Document + Signature to Receiver
======================================================================

[RECEIVER] Received document and signature
✓ Signature VERIFIED - Document is authentic and unmodified
✓ Document integrity confirmed
✓ Sender identity authenticated

======================================================================
TESTING: Tampered Document Detection
======================================================================

[ATTACKER] Modified document: This is a confidential contract agreement between Party A and Party B worth
$50,000 [TAMPERED]
✗ Signature INVALID - Tampering detected!
```

8. Students needs to conduct a data privacy audit of an organization to identify potential vulnerabilities and risks in their data privacy practices.

```
# ============================================================================
# PROGRAM 8: Data Privacy Audit Tool
# ============================================================================

import json
from datetime import datetime

class DataPrivacyAudit:
    def __init__(self, organization_name):
        self.org_name = organization_name
        self.audit_date = datetime.now().strftime("%Y-%m-%d")
        self.vulnerabilities = []
        self.risks = []
        self.compliance_score = 0

    def audit_data_collection(self):
        """Audit data collection practices"""
        print("\n" + "=" * 70)
        print("1. DATA COLLECTION AUDIT")
        print("=" * 70)

        questions = [
            "Is user consent obtained before data collection? (yes/no): ",
            "Is collected data minimized to necessity only? (yes/no): ",
            "Are data retention periods defined? (yes/no): ",
            "Is purpose of data collection clearly stated? (yes/no): "
        ]

        score = 0
        for q in questions:
            answer = input(q).lower()
            if answer == 'yes':
                score += 25
            else:
                self.vulnerabilities.append(q.replace(" (yes/no): ", ""))

        return score

    def audit_data_storage(self):
        """Audit data storage security"""
        print("\n" + "=" * 70)
        print("2. DATA STORAGE SECURITY AUDIT")
        print("=" * 70)

        questions = [
            "Is data encrypted at rest? (yes/no): ",
            "Are access controls implemented? (yes/no): ",
            "Are databases password-protected? (yes/no): ",
            "Is data backed up regularly? (yes/no): "
        ]
```

```
        score = 0
        for q in questions:
            answer = input(q).lower()
            if answer == 'yes':
                score += 25
            else:
                self.vulnerabilities.append(q.replace(" (yes/no): ", ""))

        return score

    def audit_data_transmission(self):
        """Audit data transmission security"""
        print("\n" + "=" * 70)
        print("3. DATA TRANSMISSION SECURITY AUDIT")
        print("=" * 70)

        questions = [
            "Is HTTPS/TLS used for data transmission? (yes/no): ",
            "Are APIs secured with authentication? (yes/no): ",
            "Is data encrypted during transmission? (yes/no): ",
            "Are secure protocols used (SSH, SFTP)? (yes/no): "
        ]

        score = 0
        for q in questions:
            answer = input(q).lower()
            if answer == 'yes':
                score += 25
            else:
                self.vulnerabilities.append(q.replace(" (yes/no): ", ""))

        return score

    def audit_user_rights(self):
        """Audit user rights implementation"""
        print("\n" + "=" * 70)
        print("4. USER RIGHTS & PRIVACY AUDIT")
        print("=" * 70)

        questions = [
            "Can users access their personal data? (yes/no): ",
            "Can users request data deletion? (yes/no): ",
            "Is privacy policy clearly stated? (yes/no): ",
            "Are users notified of data breaches? (yes/no): "
        ]

        score = 0
        for q in questions:
            answer = input(q).lower()
            if answer == 'yes':
                score += 25
            else:
                self.vulnerabilities.append(q.replace(" (yes/no): ", ""))
```

```
        return score

    def assess_risks(self):
        """Assess risk levels based on vulnerabilities"""
        vuln_count = len(self.vulnerabilities)

        if vuln_count == 0:
            self.risks.append("LOW RISK: Excellent privacy practices")
        elif vuln_count <= 4:
            self.risks.append("MEDIUM RISK: Some improvements needed")
        elif vuln_count <= 8:
            self.risks.append("HIGH RISK: Significant vulnerabilities detected")
        else:
            self.risks.append("CRITICAL RISK: Major privacy concerns")

    def generate_report(self):
        """Generate audit report"""
        print("\n" + "=" * 70)
        print("DATA PRIVACY AUDIT REPORT")
        print("=" * 70)
        print(f"\nOrganization: {self.org_name}")
        print(f"Audit Date: {self.audit_date}")
        print(f"\nOverall Compliance Score: {self.compliance_score}%")

        print("\n--- IDENTIFIED VULNERABILITIES ---")
        if self.vulnerabilities:
            for i, vuln in enumerate(self.vulnerabilities, 1):
                print(f"{i}. {vuln}")
        else:
            print("No vulnerabilities found")

        print("\n--- RISK ASSESSMENT ---")
        for risk in self.risks:
            print(f"• {risk}")

        print("\n--- RECOMMENDATIONS ---")
        if self.compliance_score < 50:
            print("• URGENT: Implement basic privacy measures immediately")
            print("• Conduct employee training on data privacy")
            print("• Review and update privacy policies")
        elif self.compliance_score < 75:
            print("• Strengthen data encryption practices")
            print("• Improve access control mechanisms")
            print("• Regular security audits recommended")
        else:
            print("• Maintain current privacy standards")
            print("• Continue regular compliance reviews")
            print("• Stay updated with privacy regulations")

        # Save report to file
        report = {
            "organization": self.org_name,
            "audit_date": self.audit_date,
            "compliance_score": self.compliance_score,
```

```
            "vulnerabilities": self.vulnerabilities,
            "risks": self.risks
        }

        filename = f"privacy_audit_{self.org_name.replace(' ', '_')}_{self.audit_date}.json"
        with open(filename, 'w') as f:
            json.dump(report, f, indent=2)

        print(f"\nReport saved to: {filename}")

# Run the audit
print("\n" + "=" * 70)
print("DATA PRIVACY AUDIT SYSTEM")
print("=" * 70)

org_name = input("\nEnter organization name: ")
audit = DataPrivacyAudit(org_name)

print("\nThis audit will assess your organization's data privacy practices.")
print("Answer the following questions honestly (yes/no).\n")

# Conduct audits
score1 = audit.audit_data_collection()
score2 = audit.audit_data_storage()
score3 = audit.audit_data_transmission()
score4 = audit.audit_user_rights()

# Calculate total score
audit.compliance_score = (score1 + score2 + score3 + score4) // 4

# Assess risks
audit.assess_risks()

# Generate report
audit.generate_report()
```

OUTPUT:

```
================================================================================
DATA PRIVACY AUDIT SYSTEM
================================================================================

Enter organization name: SecureBank Corp

This audit will assess your organization's data privacy practices.
Answer the following questions honestly (yes/no).


================================================================================
1. DATA COLLECTION AUDIT
================================================================================
Is user consent obtained before data collection? (yes/no): yes
Is collected data minimized to necessity only? (yes/no): yes
Are data retention periods defined? (yes/no): yes
Is purpose of data collection clearly stated? (yes/no): yes

================================================================================
2. DATA STORAGE SECURITY AUDIT
================================================================================
Is data encrypted at rest? (yes/no): yes
Are access controls implemented? (yes/no): yes
Are databases password-protected? (yes/no): yes
Is data backed up regularly? (yes/no): yes

================================================================================
3. DATA TRANSMISSION SECURITY AUDIT
================================================================================
Is HTTPS/TLS used for data transmission? (yes/no): yes
Are APIs secured with authentication? (yes/no): yes
Is data encrypted during transmission? (yes/no): yes
Are secure protocols used (SSH, SFTP)? (yes/no): yes
```

```
================================================================================
4. USER RIGHTS & PRIVACY AUDIT
================================================================================
Can users access their personal data? (yes/no): yes
Can users request data deletion? (yes/no): yes
Is privacy policy clearly stated? (yes/no): yes
Are users notified of data breaches? (yes/no): yes

================================================================================
DATA PRIVACY AUDIT REPORT
================================================================================

Organization: SecureBank Corp
Audit Date: 2025-11-16

Overall Compliance Score: 100%

--- IDENTIFIED VULNERABILITIES ---
No vulnerabilities found

--- RISK ASSESSMENT ---
• LOW RISK: Excellent privacy practices

--- RECOMMENDATIONS ---
• Maintain current privacy standards
• Continue regular compliance reviews
• Stay updated with privacy regulations

Report saved to: privacy_audit_SecureBank_Corp_2025-11-16.json
```

.json file where the report is locally saved: